

JAVA 17 OCP · YDS STUDY SERIES

Unit 14 · I/O — Technical Memory Notes

Teknik İngilizceyi bağlam içinde öğren · kısa tekrarlarla kalıcılaştır

Bu not, [ana çift dilli dersin](#) `File`, `Path`, NIO.2, I/O stream, serialization, `Console`, file attribute ve directory traversal kurallarını Java 17 sınav kararlarına dönüştürür. Sorular özgün OCP tarzı çalışma sorularıdır; gerçek sınav sorusu değildir.

[Vocabulary](#) · [Grammar notes](#)

1. İlk karar: `File`, `Path` veya `Files`

İhtiyaç	Temel API	Kritik ayrıntı
Legacy path reference	<code>java.io.File</code>	Object oluşturmak disk erişimi yapmaz
Modern path value	<code>java.nio.file.Path</code>	Interface ve immutable value'dur
NIO.2 file operation	<code>java.nio.file.Files</code>	Çoğu method static'tir
File-system provider	<code>FileSystem</code> , <code>FileSystems</code>	<code>FileSystem</code> abstract class'tir

```
File legacy = new File("/data/zoo.txt");
Path modern = Path.of("/data", "zoo.txt");

File again = modern.toFile();
Path againModern = legacy.toPath();
```

Bu satırlar yalnız reference oluşturur. `File`'in gerçekten var olup olmadığını öğrenmek için `legacy.exists()` veya `Files.exists(modern)` gibi ayrı bir operation gerekir.

`Path` interface olduğundan `new Path(...)` **does not compile**. `Path.of(...)`, `Paths.get(...)` veya `FileSystems.getDefault().getPath(...)` kullanılır.

2. Absolute, relative ve path symbol

```
/animals/bear.txt → Unix benzeri sistemde absolute
animals/bear.txt → relative
. → current directory
.. → parent directory
```

Absolute/relative kararı file-system provider'a bağlıdır. OCP sorusu açıkça Unix convention veriyorsa baştaki `/` root anlamındadır. Windows drive syntax'ını Unix provider üzerinde aynı biçimde yorumlamaya çalışma.

```
Path p = Path.of("/zoo/./bear/../food.txt");
System.out.println(p);
System.out.println(p.normalize());
```

Unix benzeri provider için:

```
/zoo/./bear/../food.txt
/zoo/food.txt
```

İlk satır `Path`'in textual value'sunu korur. `normalize()` yeni bir `Path` döndürür; original value değişmez.

3. `Path` immutable'dır

```
Path p = Path.of("/zoo/./bear");
p.normalize();
System.out.println(p);           // /zoo/./bear
p = p.normalize();
System.out.println(p);           // /zoo/bear
```

`String` gibi, result yeniden atanmazsa operation'ın etkisi kaybolur.

4. `Path` element method'ları

`Path.of("/zoo/animals/bear.txt")` için Unix benzeri provider'da:

Expression	Result
<code>getNameCount()</code>	3
<code>getName(0)</code>	zoo
<code>getName(2)</code>	bear.txt
<code>getFileName()</code>	bear.txt
<code>getParent()</code>	/zoo/animals
<code>getRoot()</code>	/
<code>subpath(0, 2)</code>	zoo/animals

Root, name element sayılmaz. `getName()` zero-based; `subpath(begin, end)` begin-inclusive ve end-exclusive çalışır. Geçersiz index runtime'da `IllegalArgumentException` üretir.

5. `resolve()`, `relativize()` ve `normalize()`

`resolve()`

```
Path base = Path.of("/zoo/animals");
System.out.println(base.resolve("bear.txt")); // /zoo/animals/bear.txt
System.out.println(base.resolve("/food.txt")); // /food.txt
```

Argument absolute ise base tamamen yok sayılır.

`relativize()`

```
Path a = Path.of("/fish/shark");
Path b = Path.of("/fish/clown");
System.out.println(a.relativize(b));           // ../clown
```

Genel sınav kuralı: İki path compatible olmalıdır. Bir absolute ile bir relative path'i relativize etmek `IllegalArgumentException` üretir.

```
normalize()
```

`normalize()` . ve mümkün olan `name/..` çiftlerini textual olarak temizler. File system'e erişmez ve symbolic link izlemez. Bu nedenle symbolic link bulunan bir path'te fiziksel konumu değiştirebilecek biçimde kullanılırken dikkat gerekir.

6. `toAbsolutePath()` ile `toRealPath()`

Method	File system gerekir mi?	Symbolic link	Path symbol
<code>toAbsolutePath()</code>	Genellikle hayır	İzlemez	Koruyabilir
<code>normalize()</code>	Hayır	İzlemez	Textual olarak temizler
<code>toRealPath()</code>	Evet	Varsayılan olarak izler	Temizler

`toRealPath()` target yoksa checked `IOException` alt türü `NoSuchFileException` üretebilir. Symbolic link'ler varsayılan olarak çözülür; `LinkOption.NOFOLLOW_LINKS` verilirse çözülmez. Provider-specific edge case'lerin bulunabileceğini unutma.

7. Ortak `File` ve `Files` operation'ları

Legacy <code>File</code>	Modern <code>Files</code>	Not
<code>exists()</code>	<code>exists(Path, ...)</code>	boolean; checked exception declare etmez
<code>isDirectory()</code>	<code>isDirectory(Path, ...)</code>	boolean
<code>isFile()</code>	<code>isRegularFile(Path, ...)</code>	boolean
<code>length()</code>	<code>size(Path)</code>	<code>Files.size()</code> <code>IOException</code> declare eder
<code>lastModified()</code>	<code>getLastModifiedTime(Path, ...)</code>	NIO.2 <code>FileTime</code> döndürür
<code>delete()</code>	<code>delete(Path)</code> / <code>deleteIfExists(Path)</code>	Return/exception modeli farklı
<code>listFiles()</code>	<code>list(Path)</code>	NIO.2 result bir <code>Stream<Path></code>

`Files.list()`, `Files.walk()`, `Files.find()` ve `Files.lines()` file-system resource açabilir. Returned stream try-with-resources ile kapatılmalıdır.

```
try (Stream<Path> entries = Files.list(Path.of("/zoo"))) {
    entries.forEach(System.out::println);
}
```

8. `LinkOption.NOFOLLOW_LINKS`

`Files.exists()`, `isDirectory()`, `isRegularFile()` gibi method'lar varsayılan olarak symbolic link target'ını inceler. `NOFOLLOW_LINKS` verildiğinde link'in kendisini değerlendirir.

```
boolean targetExists = Files.exists(link);
boolean linkExists =
    Files.exists(link, LinkOption.NOFOLLOW_LINKS);
```

`Files.isSymbolicLink(path)` ayrıca link'in kendisini test eder ve checked exception declare etmez.

9. Directory oluşturma

```
Files.createDirectory(Path.of("/zoo/animals"));
Files.createDirectories(Path.of("/zoo/animals/bears"));
```

- `createDirectory()` yalnız son directory'yi oluşturur. Parent yoksa veya target zaten varsa exception üretir.
- `createDirectories()` eksik parent'ları da oluşturur. Target directory zaten varsa normal olarak exception üretmez.
- İkisi de `Path` döndürür ve `IOException` declare eder.

Legacy karşılık:

```
new File("/zoo/animals").mkdir();
new File("/zoo/animals/bears").mkdirs();
```

Legacy method'lar success bilgisini boolean ile bildirir.

10. Copy

Path → Path

```
Files.copy(source, target);
Files.copy(source, target, StandardCopyOption.REPLACE_EXISTING);
```

Default durumda target varsa `FileAlreadyExistsException` oluşabilir. `COPY_ATTRIBUTES` mümkünse attributes'ı kopyalar. `NOFOLLOW_LINKS`, source symbolic link ise target'ı izlemek yerine link'in kendisini kopyalama davranışını ister.

Bir directory path'ini `Files.copy()` ile kopyalamak recursive tree copy değildir. Directory entry oluşturulabilir; içindeki children otomatik kopyalanmaz.

Stream overload'ları

```
long count = Files.copy(sourcePath, outputStream);
long count2 = Files.copy(inputStream, targetPath);
```

Overload yönünü argument order'dan oku. `InputStream → Path` overload'ında target zaten varsa `REPLACE_EXISTING` gerekir.

11. Move

```
Files.move(source, target);
Files.move(source, target, StandardCopyOption.REPLACE_EXISTING);
Files.move(source, target, StandardCopyOption.ATOMIC_MOVE);
```

- Target, destination file'in tamamıdır; “directory içine koy” anlamına gelmez.
- `ATOMIC_MOVE`, observer'in incomplete intermediate state görmemesini ister.
- Provider atomic move desteklemiyorsa `AtomicMoveNotSupportedException` oluşabilir.
- `ATOMIC_MOVE` varsa diğer copy option'lar Java 17 contract'ına göre yok sayılır. Atomic move'un nasıl gerçekleştirildiği implementation-specific'tir.

12. Delete

```
Files.delete(path); // void; yoksa exception
Files.deleteIfExists(path); // boolean; yoksa false
```

İki method da `IOException` declare eder. Nonempty directory için `DirectoryNotEmptyException` oluşabilir. Path symbolic link ise target değil, link silinir. NIO.2'de tek çağrıyla bütün directory tree'yi silen bir method yoktur.

13. File identity ve content comparison

```
Files.isSameFile(p1, p2);
Files.mismatch(p1, p2);
```

- `Path.equals()` yalnız provider'ın path value equality kuralını uygular; disk identity kontrolü değildir.
- `Files.isSameFile()` file-system'e başvurabilir; symbolic link ve redundant symbols sonrasında aynı file'ı gösterebilir.
- `Files.mismatch()` content aynıysa `-1`; farklıysa ilk farklı byte'ın zero-based position'ını döndürür.

14. Stream nomenclature karar ağacı

```
Data unit?
├─ byte      → InputStream / OutputStream
└─ character → Reader / Writer

Direction?
├─ read      → InputStream / Reader
└─ write     → OutputStream / Writer

Source/sink'e doğrudan mı?
├─ yes       → low-level (FileInputStream, FileReader, ...)
└─ başka stream'i wrap eder → high-level
```

Class adını soldan sağa çöz:

Buffered | Object | Data | Print + InputStream | OutputStream | Reader | Writer
işlev data türü ve yön

15. Dört abstract base class

Base class	Unit	Direction
<code>InputStream</code>	byte	input
<code>OutputStream</code>	byte	output
<code>Reader</code>	char	input
<code>Writer</code>	char	output

Byte ve character hierarchy'leri constructor chain içinde karıştırılmaz:

```
new BufferedReader(new FileReader("zoo.txt")); // compiles
new BufferedInputStream(new FileInputStream("zoo")); // compiles
new BufferedInputStream(new FileReader("zoo.txt")); // DOES NOT COMPILE
```

16. Bridge stream'ler

`InputStreamReader`, byte input'u character input'a decode eder. `OutputStreamWriter`, character output'u byte output'a encode eder.

```
Reader reader = new InputStreamReader(
    new FileInputStream("zoo.txt"),
    StandardCharsets.UTF_8
);
```

Encoding'in açık yazılması platform default'una bağımlılığı önler.

17. Buffer ile doğru copy loop

```
void copy(InputStream in, OutputStream out) throws IOException {
    byte[] buffer = new byte[8192];
    int length;
    while ((length = in.read(buffer)) != -1) {
        out.write(buffer, 0, length);
    }
}
```

OCP tuzağı:

```
while (in.read(buffer) != -1) {
    out.write(buffer); // son turda stale byte'ları da yazabilir
}
```

`read(buffer)` sonucu kullanılmazsa son buffer tam dolmadığında output bozulur. `read()` için `-1` EOF'tur; `0`, EOF anlamına gelmez.

18. `flush()` ve `close()`

- `flush()` buffered output'u downstream target'a gönderir.
- `close()` resource'u kapatır. Bazı concrete buffered writer/output implementation'ları `close` sırasında pending output'u yazar; ancak `Flushable` interface'inin kendisi `close()` davranışı tanımlamaz.
- try-with-resources resources'ı declaration'ın ters sırasında kapatır.
- Outer wrapper'ı kapatmak normal olarak wrapped stream'i de kapatır.

```
try (var out = new BufferedWriter(new FileWriter("zoo.txt"))) {
    out.write("lion");
}
```

`close()` sonrası yeniden write etmek compile-time error değil, runtime `IOException` problemidir.

19. Files convenience method'ları

Method	Eager/lazy	Result
<code>readString(Path)</code>	eager	<code>String</code>
<code>readAllBytes(Path)</code>	eager	<code>byte[]</code>
<code>readAllLines(Path)</code>	eager	<code>List<String></code>
<code>lines(Path)</code>	lazy	<code>Stream<String></code>
<code>writeString(Path, ...)</code>	eager write	<code>Path</code>
<code>write(Path, byte[]/Iterable, ...)</code>	eager write	<code>Path</code>
<code>newBufferedReader(Path)</code>	resource	<code>BufferedReader</code>
<code>newBufferedWriter(Path, ...)</code>	resource	<code>BufferedWriter</code>

Large file için `readAllLines()` tüm içeriği memory'ye taşır; `Files.lines()` veya buffered streaming daha uygundur. `Files.lines()` result'ını kapat.

`InputStream.read()` için `-1`, geçici olarak “byte hazır değil” değil **end-of-stream (EOF)** anlamındadır. `InputStream.readAllBytes()` da Java 17 instance method'ıdır:

```
byte[] content = input.readAllBytes();
```

20. Serialization temel contract

Bir object graph'ın serialize edilebilmesi için:

- 1 Runtime class `Serializable` implement etmelidir.
- 2 Non-`static`, non-`transient` primitive field'lar doğrudan serialize edilir.
- 3 Non-`static`, non-`transient` reference field value'ları `null` veya serializable olmalı; bu kural reachable object graph boyunca sürmelidir.

`Serializable` marker interface'tir; abstract method içermez.

```
class Animal implements Serializable {
    private static final long serialVersionUID = 1L;
    private String name;
    private transient int cache;
    private static String category;
}
```

- `name` serialize edilir.
- `cache` serialize edilmez; deserialization'da default `0`.
- `category` object state değil class state'tir; stream'e instance field olarak yazılmaz.
- `serialVersionUID` önerilir fakat explicit declare edilmesi zorunlu değildir.

`serialVersionUID`, "class'ta herhangi bir değişiklik oldu, bir artır" counter'ı değildir. Eski serialized form ile compatibility bilinçli olarak bozulacaksa değiştirilir; compatible evolution sırasında aynı tutulabilir.

21. `ObjectOutputStream` ve `ObjectInputStream`

```
try (var out = new ObjectOutputStream(
    new BufferedOutputStream(
        new FileOutputStream("animals.bin")))) {
    out.writeObject(animal);
}

try (var in = new ObjectInputStream(
    new BufferedInputStream(
        new FileInputStream("animals.bin")))) {
    Animal copy = (Animal) in.readObject();
}
```

`readObject()` return type `Object` olduğundan cast gerekir ve `ClassNotFoundException` declare eder. Stream sonuna kadar object okumada `null` sentinel güvenilir değildir; `null` da geçerli serialized value olabilir. Kaynağın formatını count, sentinel contract veya `EOFException` stratejisiyle tasarla.

22. Deserialization sırasında construction

`Serializable` class'ın constructor'ı ve instance initializer'ı çalıştırılmaz. Hierarchy'de bulunan ilk non-serializable superclass'ın no-arg constructor'ı çalıştırılır.

```
class Parent {
    String label;
    Parent() { label = "parent"; }
}

class Child extends Parent implements Serializable {
    transient int count = 7;
    Child() { label = "child"; }
}
```

Bir `Child` deserialize edildiğinde `Parent()` çalışır; `Child()` ve `count = 7` initializer'ı çalışmaz. `count` değeri 0 olur.

İlk non-serializable superclass'ın accessible no-arg constructor'ı yoksa deserialization `InvalidClassException` ile başarısız olabilir.

23. Record ve serialization

Record otomatik olarak `Serializable` değildir:

```
record Point(int x, int y) {} // serializable değil
record SavedPoint(int x, int y) implements Serializable {}
```

Record serializable olduğunda serialization mekanizmasının construction ayrıntıları ordinary serializable class'tan farklıdır; sınav sorusunda verilen type declaration'ı dikkatle oku.

Özellikle serializable record deserialization'ında canonical constructor çağrılır. Ordinary serializable class için “constructor çalışmaz” kuralını record'a körlemesine uygulama.

24. Standard stream'ler

Reference	Static type	Yön
<code>System.in</code>	<code>InputStream</code>	input
<code>System.out</code>	<code>PrintStream</code>	output
<code>System.err</code>	<code>PrintStream</code>	output

`System.out` veya `System.err` kapatılırsa sonraki print operation'ları error'ı sessizce kaydedebilir; `PrintStream` çoğu write error'ını exception olarak fırlatmak yerine `checkError()` üzerinden bildirir. Standard stream'leri uygulama kodunda kapatmamak genel doğru yaklaşımdır.

Kaynakta kapanan standard stream'in “permanently unavailable” olduğu söylenir; bu ordinary program akışı için yararlı bir uyarıdır fakat mutlak API kuralı değildir. `System.setIn()`, `System.setOut()` ve `System.setErr()` uygun permission/environment altında reference'ları yeni stream'lerle değiştirebilir.

25. Console

```
Console console = System.console();
if (console == null) {
    System.out.println("Console unavailable");
} else {
    String user = console.readLine("User: ");
    char[] password = console.readPassword("Password: ");
}
```

- `System.console()` null dönebilir; IDE ve redirected environment'larda sık görülür.
- `Console` constructor'ı accessible değildir; `new Console()` derlenmez.

- `readPassword()` `char[]` döndürür; asıl güvenlik avantajı String pool iddiası değil, iş bitince array'in overwrite edilebilmesidir.
- `Console.writer()` `PrintWriter`, `Console.reader()` `Reader` döndürür.
- `format()` ve `printf()` `Console` döndürerek chaining sağlar.

26. `mark()`, `reset()` ve `skip()`

```
if (reader.markSupported()) {
    reader.mark(100);
    int first = reader.read();
    reader.reset();
}
```

- Her input stream `mark()` desteklemez. Mark desteklemeyen base implementation'da `mark()` no-op olabilir; `reset()` ise `IOException` üretebilir.
- `mark(readAheadLimit)` verilen limit içinde geri dönme isteğidir; garanti ayrıntıları concrete stream'e bağlıdır.
- `reset()` valid mark yoksa `IOException` üretebilir.
- `skip(n)` actual skipped count döndürür; return value istenen `n` olmak zorunda değildir.

Kaynak Review Question 18, `skip(2)` çağrılarının tam iki character atladığını varsayar. `Reader` type'ı unspecified bırakıldığından Java API contract'ı açısından return value daha küçük olabilir. Resmî C cevabı, mark destekleyen ve requested miktarı atlayan conventional reader varsayımıyla geçerlidir.

27. `available()` ne anlatır?

`InputStream.available()` bütün stream uzunluğu veya remaining byte count garantisi değildir. Blocking olmadan okunabileceği tahmin edilen byte sayısını döndürür:

```
int immediatelyReadable = input.available();
```

0, kesin EOF demek değildir. EOF kararı `read() == -1` ile verilir.

28. File attributes

Basit boolean probe'lar:

```
Files.isDirectory(path);
Files.isRegularFile(path);
Files.isSymbolicLink(path);
Files.isReadable(path);
Files.isWritable(path);
Files.isExecutable(path);
Files.isHidden(path); // IOException declare eder
```

Probe method'larının `false` dönmesi her zaman yalnız “target yok” anlamına gelmez; access, provider veya security koşulları da etkileyebilir.

Toplu attribute okuma:

```
BasicFileAttributes attrs =
    Files.readAttributes(path, BasicFileAttributes.class);

System.out.println(attrs.size());
System.out.println(attrs.creationTime());
System.out.println(attrs.lastModifiedTime());
```

Attribute değiştirmek için read-only snapshot değil view kullanılır:

```
BasicFileAttributeView view =
    Files.getFileAttributeView(path, BasicFileAttributeView.class);
view.setTimes(FileTime.fromMillis(0), null, null);
```

29. Attribute view aileleri

View	Amaç
<code>BasicFileAttributeView</code>	Ortak basic times/type/size
<code>DosFileAttributeView</code>	DOS hidden, archive, system, read-only
<code>PosixFileAttributeView</code>	POSIX owner/group/permissions
<code>FileOwnerAttributeView</code>	Owner
<code>UserDefinedFileAttributeView</code>	Provider user-defined metadata

Supported view file system'e bağlıdır. `Files.getFileAttributeView()` uygun view desteklenmiyorsa `null` dönebilir.

30. `Files.walk()` ve `depth`

```
try (Stream<Path> tree = Files.walk(root, 3)) {
    tree.forEach(System.out::println);
}
```

- Root depth 0'dır.
- Default traversal symbolic link izlemez.
- `FOLLOW_LINKS` verilirse cycle riski vardır; provider `FileSystemLoopException` üretebilir.
- Encounter order depth-first'tür.
- Lazy stream iteration sırasında `UncheckedIOException` görülebilir.

`maxDepth = 0` yalnız başlangıç path'ini ziyaret eder.

31. Files.find()

```
try (Stream<Path> result = Files.find(
    root,
    10,
    (path, attrs) -> attrs.isRegularFile()
    && path.toString().endsWith(".java"))) {
    result.forEach(System.out::println);
}
```

`Files.find()` `filter()` `BiPredicate<Path, BasicFileAttributes>` alır.

`Stream<Path>.filter()` ise yalnız `Predicate<Path>` alır. Bu iki signature OCP sorularında sıkça birbirine karıştırılır.

32. Files.walk() ile Files.find() karşılaştırması

Özellik	<code>walk()</code>	<code>find()</code>
Result	<code>Stream<Path></code>	<code>Stream<Path></code>
Predicate argument	Yok	<code>BiPredicate<Path, BasicFileAttributes></code>
Depth	overload ile	zorunlu
Attributes	Ayrı çağrı gerekir	Predicate'e verilir
Close gerekir	Evet	Evet

33. Kaynak metin ve Review Question errata

NIO açılımı

Kaynak NIO'yu yalnız “non-blocking I/O” diye açar. NIO ailesi yaygın biçimde **New I/O** olarak da adlandırılır ve NIO.2 `Files` operation'larının genel olarak non-blocking olduğu sonucu çıkarılamaz; file operation'ları blocking olabilir.

Recursive copy ve symbolic link

Kaynağın recursive copy örneği directory entry'lerini `Files.list()` ile gezer. Bu örnekte bir directory symbolic link, default `Files.isDirectory()` tarafından target directory olarak görülebilir. `NOFOLLOW_LINKS` veya explicit link policy olmadan genel bir “hiçbir symbolic link izlenmez” sonucu çıkarılamaz.

API ve metin düzeltmeleri

- Kaynak page 847 prose'daki `FOLLOW_LINK` tekildir. Gerçek enum sabiti `FileVisitOption.FOLLOW_LINK` tir.
- Table 14.14'teki `OuputStream` yazımı `OutputStream` olmalıdır.
- `read() == -1`, “o anda byte yok” değil EOF'tur.
- `readAllBytes()` `InputStream` instance method'ıdır.
- `serialVersionUID` her class edit'inde otomatik artırılan version counter değildir.
- Serializable record deserialization'ında canonical constructor çağrılır.
- `available()` blocking olmadan okunabileceği tahmin edilen byte miktarıdır.
- Standard stream reference'ları `System.setIn/Out/Err()` ile değiştirilebilir.
- Password için `char[]` avantajı iş bitince temizlenebilmesidir.
- Mark unsupported olduğunda `mark()` no-op olabilir; `reset()` exception üretebilir.

Question 15

Kaynak option F şu path'i kullanır:

```
Path.of("/weather/winer/snow.dat").toFile()
```

Soru ise var olduğu verilen `/weather/winter/snow.dat` file'ını sorar. `winer/winter` farkı anlamlıdır. Appendix'in resmî **B**, **E**, **F** anahtarı korunur; literal Java 17 koduna göre F başka bir path'i temsil eder ve teknik doğru küme **B**, **E** olur.

Question 17

Resmî anahtar **B**, **C**, **E** dir. Appendix açıklamasının ikinci paragrafı file-system-specific attribute'ların view'larla erişilebildiğini söyledikten sonra yanlışlıkla “option D is correct” yazar. Option D bunun tersini iddia eder. Bu bir explanation typo'sudur; resmî anahtarın kendisiyle de çelişir.

Question 18

Resmî C cevabı conventional reader'ın `skip(2)` isteğini tam karşılaması varsayımına dayanır. General `Reader.skip()` actual count döndürür; generic reader için değer kontrol edilmeden tam skip garantisi kurulamaz.

34. OCP karar kontrol listesi

Bir I/O sorusunda şu sırayla ilerle:

- 1 Static type ve exact overload'u belirle.
- 2 Path operation immutable result döndürüyor mu kontrol et.
- 3 File-system erişimi var mı, yalnız value manipulation mı ayır.
- 4 Checked `IOException` handled/declared mı kontrol et.
- 5 Symbolic link default follow davranışını yaz.
- 6 Copy/move target'in directory değil exact destination olduğunu hatırla.

- 7 Stream hierarchy byte/character ve input/output bakımından uyumlu mu bak.
- 8 `read()` result ve actual length kullanılıyor mu kontrol et.
- 9 Resource kapanıyor mu; kapandıktan sonra yeniden kullanılıyor mu bak.
- 10 Serialization'da runtime object graph ve `transient/static` alanları işaretle.
- 11 Directory traversal'da depth, follow-links ve stream close durumunu yaz.
- 12 Sonucu **does not compile**, checked/runtime exception, deterministic output veya environment-dependent result olarak sınıflandır.

35. Mini quiz

Soru 1

Kaç seçenek doğrudur?

```
Path p = Path.of("/zoo/./bear/../food.txt");
```

A. `p.normalize()` original `p`'yi değiştirir. B. `p.normalize()` file system'e erişmeden yeni `Path` döndürür. C. `p.toRealPath()` target yoksa exception üretebilir. D. `p.resolve("/lion")` Unix benzeri provider'da `/lion` verir.

Soru 2

Bu loop'taki temel hata nedir?

```
byte[] data = new byte[10];
while (in.read(data) != -1)
    out.write(data);
```

Soru 3

Hangisi **does not compile**?

A. `new BufferedReader(new FileReader("a.txt"))` B. `new BufferedInputStream(new FileInputStream("a.bin"))` C. `new BufferedReader(new FileInputStream("a.txt"))` D. `new ObjectInputStream(new FileInputStream("a.bin"))`

Soru 4

`Files.walk(root, 0)` hangi path'leri ziyaret eder?

Soru 5

Serializable subclass'ın constructor'ı deserialization sırasında çalışır mı? İlk non-serializable superclass için kural nedir?

Soru 6

```
Path target = Files.createDirectories(Path.of("/zoo"));
```

`/zoo` zaten directory ise kesin olarak `FileAlreadyExistsException` oluşur mu?

Soru 7

`System.console()` için hangi iki durum mümkündür?

A. Her zaman non-null B. `null` C. `Console` instance D. `IOException`

Soru 8

`BasicFileAttributes` üzerinden `setTimes()` çağrılabilir mi?

36. Cevaplar ve açıklamalar

- 1 **B, C, D.** `Path` immutable'dir. `normalize()` textual operation; `toRealPath()` file-system operation'dır; absolute argument `resolve()` da base'i geçersiz kılar.
- 2 `read(data)` actual count saklanmıyor. Son iteration'da buffer'ın eski bölümü de yazılabilir. `out.write(data, 0, count)` kullanılmalıdır.
- 3 **C — Does not compile.** `BufferedReader`, `Reader` ister; `FileInputStream` ise `InputStream`dir.
- 4 Yalnız başlangıç `root` path'i, yani depth 0.
- 5 `Serializable` subclass constructor/initializer çalışmaz. İlk non-serializable superclass'ın accessible no-arg constructor'ı çalışır.
- 6 Hayır. `createDirectories()` existing directory'yi kabul eder.
- 7 **B ve C.** `System.console()` environment'a göre `null` veya `Console` döndürür.
- 8 Hayır. `BasicFileAttributes` read-only snapshot'tır; `BasicFileAttributeView` kullanılmalıdır.

Kısa tekrar özeti

- `File` legacy reference; `Path` immutable modern value; `Files` static operation helper'idır.
- `normalize()` yalnız syntax, `toRealPath()` gerçek file system bilgisidir.
- Copy/move target exact destination'dır; directory tree otomatik recursive kopyalanmaz.
- Byte hierarchy ile character hierarchy'yi karıştırma.
- Buffer copy'de `read()` return count'unu kullan.
- `Files.lines/list/walk/find` stream'lerini kapat.
- `Serializable` bir marker interface'tir; runtime object graph önemlidir.
- `transient` ve `static` alanlar ordinary serialized instance state'e girmez.
- `Console` null olabilir; standard stream'leri kapatma.
- `markSupported()`, actual `skip()` count ve directory traversal depth sınavın ince ayrıntılarıdır.